

Author of this document : Arvind Yadav

Guided by : Manan Sharma



Phase 1: Foundation & Simple Text Ingestion (.txt, .csv, .md)

Objective: Establish the base software architecture, authentication, and simple linear text parsing pipelines.

Core Engineering Tasks

- **Frontend (React.js):** Build a drag-and-drop file upload zone using [react-dropzone](#). Build a processing status dashboard displaying progress rings (e.g., *Queued* $\rightarrow$ *Processing* $\rightarrow$ *Indexed*).
- **Backend (Node.js):** Set up a relational tracking database (PostgreSQL/Prisma) to track [Document](#) state and metadata.
- **Ingestion (Node.js/Python):** Write clean, native string-stream reading modules for [.txt](#), [.csv](#), and [.md](#).
- **Vector Engine:** Implement an embedding pipeline using [text-embedding-3-small](#) and sync the vectors into a vector database (e.g., Pinecone or pgvector).

✓ Acceptance Criteria

- Files under 10MB process synchronously in under 2 seconds.
- A user can run a semantic search from the React UI and see relevant text snippets ranked by distance score.
- Uploading a malformed or empty text file throws a friendly user notification rather than crashing the system.

Phase 2: Structural Document Parsing (Digital PDFs & Excel Sheets)

Objective: Parse structured layout boundaries. Ensure that complex data layouts do not lose their meaning when parsed.

🔧 Core Engineering Tasks

- **PDF Parsing Module:** Integrate open-source structural parsers like **Docling (IBM)** or **PyMuPDF4LLM** within a Python microservice to extract digital [.pdf](#) structures into a hierarchy.
- **Excel Parsing Module:** Build an Excel processing pipeline using [xlsx](#) (SheetJS) on the backend. Convert sheets into structured JSON data rows and markdown tables instead of long, flat strings.
- **Frontend Data Viewer:** Build an interactive "Document View" using Tailwind CSS where users can view the original PDF side-by-side with its newly structured Markdown representation.

✓ Acceptance Criteria

- Multi-column PDFs are read top-to-bottom, column-by-column, preventing mixed-text reading errors.
- Excel spreadsheets with multiple workbooks are completely converted into structured markdown tables without dropping edge cells or formatting values.

Phase 3: Vision AI & Multi-Modal Processing (Scanned Documents & Images)

Objective: Extract structural information from visual media (scans, screenshots, and diagrams).

Core Engineering Tasks

- **Vision-Language Model (VLM) Pipeline:** Set up a specialized Python service running **Llama 3.2 Vision** or **SmolDocling** to read un-selectable image formats (.png, .jpeg, .tiff, scanned PDFs).
- **Image Optimization Layer:** Use OpenCV on the backend to automatically deskew, sharpen, and normalize low-light snapshots or whiteboard photographs before feeding them into the vision model.
- **Table Extraction System:** Run an agentic parsing step over images containing tables to generate complete, clean markdown tables with identical column headers.

Acceptance Criteria

- Scanned text images achieve >95% accuracy compared to manual text matching.
- Blurred charts and tables successfully convert into cleanly readable markdown rows and columns, verifying 100% visual data utilization.

Phase 4: Omnichannel Client Communications Integration (WhatsApp, Skype, Google Chat)

Objective: Build an outbound communication engine that lets external chat platforms query and query your centralized knowledge base.

Core Engineering Tasks

- **WhatsApp Gateway:** Connect your backend to the official **Meta WhatsApp Cloud API**. Set up webhooks to capture incoming client messages and reply with formatted text.
- **Skype & Google Chat Adapters:** Use the Microsoft Bot Framework for Skype and Google Chat Webhooks/APIs to build identical middleware receivers.
- **The Context Router:** Route all chat incoming requests through your Phase 1-3 RAG architecture to answer client questions using document context, appending source citations to every answer.

Acceptance Criteria

- A client can send a message on WhatsApp asking: "What is our liability limit in the contract?" and receive a secure, accurate response in under 3 seconds.
- Conversations are securely isolated; a client chatting from Skype cannot access or query data uploaded by another organization's workspace.

Commercial Presentation: High-Value Features Required to Sell as an Enterprise SaaS Product

To position this platform as a high-margin business-to-business product, the software needs features that appeal directly to enterprise decision-makers (CTOs, Legal Counsel, and Security Officers).

1. Enterprise Security, Governance, and RBAC (Role-Based Access Control)

Organizations will not buy an AI tool unless they have absolute control over who views specific information.

- **Feature:** Document-Level Access Lists. Ensure the vector retrieval layer filters chunks dynamically based on the active user's Active Directory/OAuth security groups.
- **Why it sells:** Prevents general staff from querying highly sensitive executive data (like payroll spreadsheets or executive emails) while using the chatbot.

2. Traceability UI (Zero-Hallucination Audit Trails)

Enterprise users will not trust an AI blindly. They need proof.

- **Feature:** Citations with Direct Deep-Linking. In your React frontend, clicking an AI text citation opens the original document viewer and highlights the exact bounding-box snippet or spreadsheet cell where the AI found the answer.
- **Why it sells:** Builds high confidence for risk-averse teams (such as legal, finance, and compliance) by turning a black-box AI into an auditable assistant.

3. Smart Document Diffing & Version Control

Enterprises constantly update their contracts, manuals, and internal documentation.

- **Feature:** Visual Delta Inspection. A dashboard UI that highlights structural and factual differences when a user uploads a new version of a document (e.g., *"Clause 4 changed liability from \$1M to \$5M"*).
- **Why it sells:** Transforms a standard data ingestion engine into an active, high-utility file management platform.

4. Direct White-Label Bot Deployments

- **Feature:** Single-Click Channel Provisioning. Provide your clients with a sleek, low-code interface within the React dashboard where they can input their corporate WhatsApp, Skype, or Google Chat credentials to deploy specialized customer support bots instantly.
- **Why it sells:** Provides a clear, real-time ROI to executives who want to connect their internal enterprise knowledge directly to front-line customer service workflows.

Commercial Performance Targets (Your Sales Deck Matrix)

Feature Pillar	Enterprise Demand Metric	Your Technical Proof-Point
Data Security	Zero external leak vectors	Full VPC isolation & column-level database encryption
Accuracy Baseline	No hallucinations on data	Cross-Encoder Reranking providing 95% precision
Deployment Speed	Fast integration velocity	Native React widgets & Node.js plug-and-play API endpoints